**Q) Design and implement a Convolutional Neural Network (CNN) for handwritten digit classification using the MNIST dataset. Demonstrate the steps of data preprocessing, CNN model construction, training, evaluation, and prediction using TensorFlow/Keras.**

**A)**

**Input:** import tensorflow as tf

import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

print("Libraries imported successfully.")

**ouput:**

Libraries imported successfully.

**Input**: # Step 2: Load MNIST handwritten digit dataset

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training images:", X_train.shape)

print("Testing images:", X_test.shape)

# Step 3: Display one image

plt.imshow(X_train[0], cmap="gray")

plt.title("Actual Digit: " + str(y_train[0]))

plt.axis("off")

plt.show()

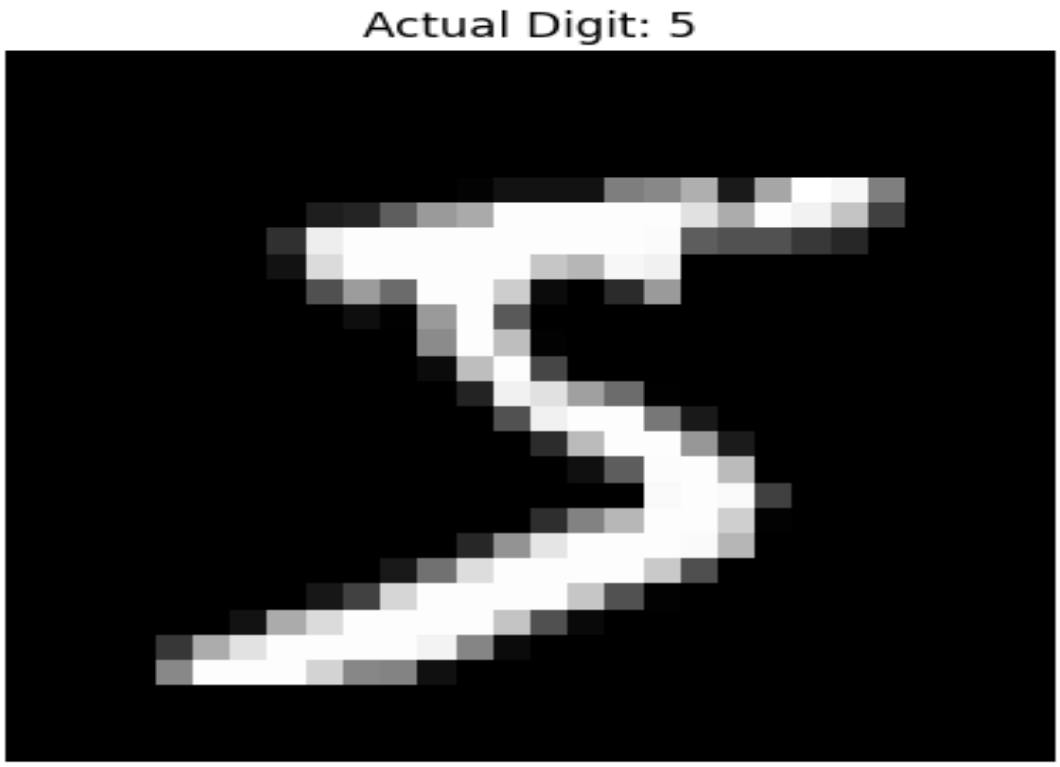
**ouput:**

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz

11490434/11490434 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Training images: (60000, 28, 28)

Testing images: (10000, 28, 28)




**Input:** # Step 4: Normalize pixel values (0-255 to 0-1)

X_train = X_train / 255.0

X_test = X_test / 255.0

# Step 5: Add channel dimension (CNN expects height, width, channel)

X_train = X_train.reshape(-1, 28, 28, 1)

X_test = X_test.reshape(-1, 28, 28, 1)

print("Data normalization and reshaping complete.")

**ouput:**

Data normalization and reshaping complete.

**Input:** # Step 6: Build the CNN model

model = Sequential([

```
    Conv2D(32, kernel_size=(3, 3), activation="relu", input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])
# Step 7: Compile the model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
# Step 8: Display model architecture
model.summary()
```

**ouput:** /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32 | 0 |
| flatten (Flatten) | (None, 5408) | 0 |
| dense (Dense) | (None, 64) | 346,176 |

| | | |
|---|---|---|
| dense_1 (Dense) | (None, 10) | 650 |

Total params: 347,146 (1.32 MB)

Trainable params: 347,146 (1.32 MB)

Non-trainable params: 0 (0.00 B)

**Input:** # Step 9: Train the model

```
model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1
)
```

**ouput:**

Epoch 1/5

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 34s 19ms/step - accuracy: 0.9427 - loss: 0.1917 - val_accuracy: 0.9820 - val_loss: 0.0698

Epoch 2/5

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 30s 18ms/step - accuracy: 0.9800 - loss: 0.0652 - val_accuracy: 0.9843 - val_loss: 0.0582

Epoch 3/5

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 30s 18ms/step - accuracy: 0.9871 - loss: 0.0430 - val_accuracy: 0.9858 - val_loss: 0.0562

Epoch 4/5

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 42s 19ms/step - accuracy: 0.9899 - loss: 0.0317 - val_accuracy: 0.9850 - val_loss: 0.0576

Epoch 5/5

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 30s 18ms/step - accuracy: 0.9924 - loss: 0.0233 - val_accuracy: 0.9873 - val_loss: 0.0572

<keras.src.callbacks.history.History at 0x7c73780ab140>

**Input:** # Step 10: Test the model

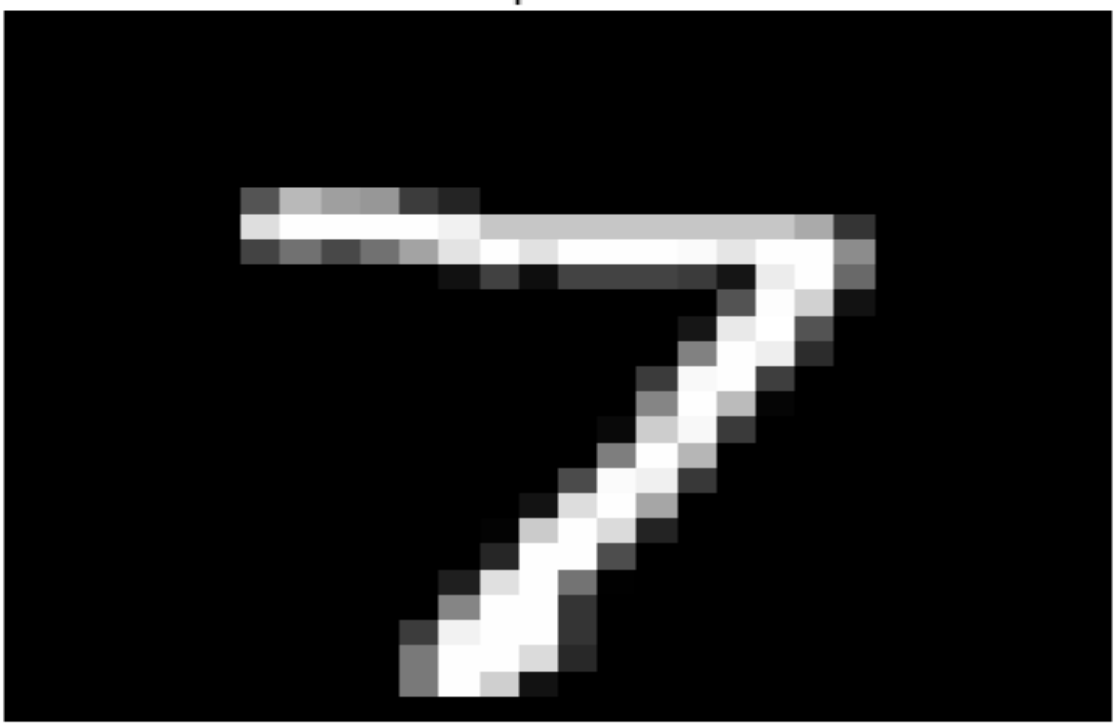
```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_accuracy)

# Step 11: Predict one test image
prediction = model.predict(X_test[0:1])
predicted_digit = prediction.argmax()
plt.imshow(X_test[0].reshape(28, 28), cmap="gray")
plt.title(
    "Actual: " + str(y_test[0]) +
    " | Predicted: " + str(predicted_digit)
)
plt.axis("off")
plt.show()
```

**output:**

**313/313** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 5ms/step - accuracy: 0.9843 - loss: 0.0493

Test Accuracy: 0.9843000173568726

**1/1** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 91ms/step


Actual: 7 | Predicted: 7